

50 Examples of the 'by' Keyword in Kotlin

The 'by' keyword in Kotlin is used for delegation - both property delegation and class delegation. This document covers various use cases.

1. Basic Class Delegation

```
interface Base {
    fun print()
}

class BaseImpl(val x: Int) : Base {
    override fun print() { println(x) }
}

class Derived(b: Base) : Base by b

val b = BaseImpl(10)
val derived = Derived(b)
derived.print() // Prints: 10
```

2. Class Delegation with Override

```
interface Base {
    fun printMessage()
    fun printMessageLine()
}

class BaseImpl(val x: Int) : Base {
    override fun printMessage() { print(x) }
    override fun printMessageLine() { println(x) }
}

class Derived(b: Base) : Base by b {
    override fun printMessage() { print("abc") }
}
```

3. Lazy Delegation

```
val lazyValue: String by lazy {  
    println("Computed!")  
    "Hello"  
}  
  
println(lazyValue) // Computed! Hello  
println(lazyValue) // Hello (no computation)
```

4. Lazy with Thread Safety Mode

```
val lazyValue: String by lazy(LazyThreadSafetyMode.PUBLICATION) {  
    expensiveComputation()  
}
```

5. Lazy with Synchronized Lock

```
val lazyValue: String by lazy(LazyThreadSafetyMode.SYNCHRONIZED) {  
    "Thread-safe initialization"  
}
```

6. Lazy with No Lock

```
val lazyValue: String by lazy(LazyThreadSafetyMode.NONE) {  
    "Not thread-safe, faster"  
}
```

7. Observable Property

```
import kotlin.properties.Delegates

var name: String by Delegates.observable("Initial") {
    prop, old, new ->
    println("$old -> $new")
}

name = "Updated" // Prints: Initial -> Updated
```

8. Observable with Custom Logic

```
var price: Int by Delegates.observable(0) {
    property, oldValue, newValue ->
    if (newValue > oldValue) {
        println("Price increased from $oldValue to $newValue")
    }
}
```

9. Vetoable Property

```
var max: Int by Delegates.vetoable(0) {
    property, oldValue, newValue ->
    newValue > oldValue
}

max = 10 // Allowed
max = 5  // Not allowed (rejected)
```

10. Vetoable with Validation

```
var age: Int by Delegates.vetoable(0) { _, _, new ->
    new >= 0 && new <= 150
}
```

11. Storing Properties in a Map

```
class User(map: Map<String, Any?>) {
    val name: String by map
    val age: Int by map
}

val user = User(mapOf(
    "name" to "John",
    "age" to 25
))
```

12. Mutable Map Delegation

```
class MutableUser(map: MutableMap<String, Any?>) {
    var name: String by map
    var age: Int by map
}

val map = mutableMapOf<String, Any?>(
    "name" to "Jane",
    "age" to 30
)

val user = MutableUser(map)
user.age = 31
```

13. Map Delegation with Default Values

```
class Config(map: Map<String, Any?>) {  
    val timeout: Int by map  
    val retries: Int by map  
}  
  
val config = Config(mapOf(  
    "timeout" to 5000,  
    "retries" to 3  
))
```

14. NotNull Property

```
import kotlin.properties.Delegates  
  
var notNullValue: String by Delegates.notNull<String>()  
  
// notNullValue = null // Compilation error  
notNullValue = "Initialized"  
println(notNullValue)
```

15. NotNull with Late Init Pattern

```
class Service {  
    var config: Config by Delegates.notNull()  
  
    fun initialize(cfg: Config) {  
        config = cfg  
    }  
}
```

16. Custom Property Delegate

```
import kotlin.reflect.KProperty

class StringDelegate {
    operator fun getValue(thisRef: Any?,
                          property: KProperty<*>): String {
        return "$thisRef, delegated '${property.name}'"
    }

    operator fun setValue(thisRef: Any?,
                          property: KProperty<*>, value: String) {
        println("$value assigned to '${property.name}'")
    }
}

var s: String by StringDelegate()
```

17. Custom Delegate with State

```
class Counter {
    private var count = 0

    operator fun getValue(thisRef: Any?,
                          property: KProperty<*>): Int {
        return count++
    }

    operator fun setValue(thisRef: Any?,
                          property: KProperty<*>, value: Int) {
        count = value
    }
}

var counter: Int by Counter()
```

18. Custom Delegate with Validation

```
class PositiveInt {  
    private var value: Int = 0  
  
    operator fun getValue(thisRef: Any?,  
                           property: KProperty<*>): Int = value  
  
    operator fun setValue(thisRef: Any?,  
                           property: KProperty<*>, newValue: Int) {  
        value = if (newValue > 0) newValue else 0  
    }  
}  
  
var positiveNumber: Int by PositiveInt()
```

19. Custom Delegate for Caching

```
class Cached<T>(val initializer: () -> T) {
    private var cached: T? = null

    operator fun getValue(thisRef: Any?,
                          property: KProperty<*>): T {
        if (cached == null) {
            cached = initializer()
        }
        return cached!!
    }
}

val cachedValue: String by Cached {
    expensiveOperation()
}
```

20. Custom Delegate with Logging

```
class Logged<T>(private var value: T) {
    operator fun getValue(thisRef: Any?,
                          property: KProperty<*>): T {
        println("Getting ${property.name}")
        return value
    }

    operator fun setValue(thisRef: Any?,
                          property: KProperty<*>, newValue: T) {
        println("Setting ${property.name} to $newValue")
        value = newValue
    }
}

var loggedProperty: String by Logged("initial")
```

21. Multiple Interface Delegation

```
interface A {  
    fun funcA()  
}  
  
interface B {  
    fun funcB()  
}  
  
class AImpl : A {  
    override fun funcA() = println("A")  
}  
  
class BImpl : B {  
    override fun funcB() = println("B")  
}  
  
class C(a: A, b: B) : A by a, B by b  
  
val c = C(AImpl(), BImpl())
```

22. Selective Method Override with Delegation

```
interface Repository {
    fun save()
    fun load()
}

class DefaultRepo : Repository {
    override fun save() = println("Saving")
    override fun load() = println("Loading")
}

class CachedRepo(repo: Repository) : Repository by repo {
    override fun load() {
        println("Checking cache first")
        // Custom logic
    }
}
```

23. Delegation with Generic Interface

```
interface Container<T> {
    fun add(item: T)
    fun get(): T
}

class ListContainer<T> : Container<T> {
    private val list = mutableListOf<T>()
    override fun add(item: T) { list.add(item) }
    override fun get(): T = list.last()
}

class LoggedContainer<T>(
    container: Container<T>
) : Container<T> by container
```

24. Delegation Chain

```
interface Service {  
    fun execute()  
}  
  
class BaseService : Service {  
    override fun execute() = println("Executing")  
}  
  
class LoggedService(s: Service) : Service by s {  
    override fun execute() {  
        println("Logging...")  
        // Calls delegated implementation  
    }  
}  
  
class CachedService(s: Service) : Service by s
```

25. Delegation with Companion Object

```
interface Factory {  
    fun create(): String  
}  
  
class DefaultFactory : Factory {  
    override fun create() = "Default"  
}  
  
class Component(factory: Factory) : Factory by factory {  
    companion object : Factory by DefaultFactory()  
}
```

26. Lazy with Custom_INITIALIZER

```
val config by lazy {  
    loadConfigFromFile()  
}  
  
fun loadConfigFromFile(): Config {  
    // Complex initialization  
    return Config()  
}
```

27. Lazy Property in Data Class

```
data class User(val id: Int) {  
    val profile: Profile by lazy {  
        fetchProfileFromDB(id)  
    }  
}
```

28. Multiple Lazy Properties

```
class Application {  
    val database by lazy { Database() }  
    val cache by lazy { Cache() }  
    val logger by lazy { Logger() }  
}
```

29. Lazy with Exception Handling

```
val resource by lazy {  
    try {  
        loadResource()  
    } catch (e: Exception) {  
        defaultResource()  
    }  
}
```

30. Lazy Circular Dependency Prevention

```
class ServiceA {  
    val serviceB by lazy { ServiceB() }  
}  
  
class ServiceB {  
    val serviceA by lazy { ServiceA() }  
}
```

31. Observable with Multiple Observers

```
var status: String by Delegates.observable("idle") {
    _, old, new ->
    notifyObservers(old, new)
    updateUI(new)
    logStatusChange(old, new)
}
```

32. Observable Property Chain

```
var temperature: Int by Delegates.observable(20) {
    _, _, new ->
    if (new > 30) {
        alarm = true
    }
}

var alarm: Boolean by Delegates.observable(false) {
    _, _, new ->
    if (new) triggerAlert()
}
```

33. Observable with Conditional Updates

```
var value: Int by Delegates.observable(0) {
    prop, old, new ->
    if (new % 2 == 0) {
        println("Even: $new")
    } else {
        println("Odd: $new")
    }
}
```

34. Provider Delegate Pattern

```
class ResourceProvider {
    operator fun provideDelegate(
        thisRef: Any?,
        prop: KProperty<*>
    ): ReadOnlyProperty<Any?, String> {
        checkProperty(prop)
        return object : ReadOnlyProperty<Any?, String> {
            override fun getValue(
                thisRef: Any?,
                property: KProperty<*>
            ) = "Resource"
        }
    }
}

val resource: String by ResourceProvider()
```

35. Provider with Registration

```
class Registry {
    private val properties = mutableListOf<String>()

    operator fun provideDelegate(
        thisRef: Any?,
        prop: KProperty<*>
    ): ReadOnlyProperty<Any?, String> {
        properties.add(prop.name)
        return object : ReadOnlyProperty<Any?, String> {
            override fun getValue(
                thisRef: Any?,
                property: KProperty<*>
            ) = prop.name
        }
    }
}
```

```
    }  
}
```

36. View Binding Delegate (Android)

```
val textView: TextView by lazy {  
    findViewById(R.id.textView)  
}
```

37. ViewModel Delegate (Android)

```
private val viewModel: MyViewModel by viewModels()
```

38. Activity Result Delegate (Android)

```
val launcher by registerForActivityResult(  
    ActivityResultContracts.StartActivityForResult()  
) { result ->  
    // Handle result  
}
```

39. Preferences Delegate (Android)

```
var theme: String by preferences.string(  
    key = "theme",  
    defaultValue = "light"  
)
```

40. ReadOnlyProperty Interface

```
import kotlin.properties.ReadOnlyProperty
import kotlin.reflect.KProperty

class FixedValue<T>(private val value: T) :
    ReadOnlyProperty<Any?, T> {
    override fun getValue(
        thisRef: Any?,
        property: KProperty<*>
    ): T = value
}

val constant: Int by FixedValue(42)
```

41. ReadWriteProperty Interface

```
import kotlin.properties.ReadWriteProperty

class Uppercase : ReadWriteProperty<Any?, String> {
    private var value = ""

    override fun getValue(
        thisRef: Any?,
        property: KProperty<*>
    ) = value

    override fun setValue(
        thisRef: Any?,
        property: KProperty<*>,
        newValue: String
    ) {
        value = newValue.uppercase()
    }
}
```

```
var text: String by Uppercase()
```

42. Delegation in Object

```
object Config {  
    val timeout: Int by lazy {  
        System.getProperty("timeout").toInt()  
    }  
}
```

43. Delegation in Companion Object

```
class Database {  
    companion object {  
        val instance: Database by lazy {  
            Database()  
        }  
    }  
}
```

44. Top-Level Property Delegation

```
val globalConfig: Map<String, String> by lazy {  
    loadConfiguration()  
}  
  
fun loadConfiguration(): Map<String, String> {  
    return mapOf("env" to "prod")  
}
```

45. Local Variable Delegation

```
fun process() {  
    val result by lazy {  
        expensiveComputation()  
    }  
  
    if (shouldCompute) {  
        println(result)  
    }  
}
```

46. Delegation with Generics

```
class Wrapper<T>(private val value: T) {
    operator fun getValue(
        thisRef: Any?,
        property: KProperty<*>
    ): T = value
}

val wrappedInt: Int by Wrapper(10)
val wrappedString: String by Wrapper("Hello")
```

47. Delegation with Type Parameters

```
interface Converter<in T, out R> {
    fun convert(value: T): R
}

class StringToIntConverter : Converter<String, Int> {
    override fun convert(value: String) = value.toInt()
}

class ConvertedProperty<T, R>(
    private val converter: Converter<T, R>
) {
    private lateinit var original: T

    operator fun getValue(
        thisRef: Any?,
        property: KProperty<*>
    ): R = converter.convert(original)

    operator fun setValue(
        thisRef: Any?,
        property: KProperty<*>,
        value: T
    ) {
        original = value
    }
}
```

```
    ) {  
        original = value  
    }  
}
```

48. Nullable Property Delegation

```
var nullableValue: String? by Delegates.observable(null) {  
    _, old, new ->  
    println("Changed from $old to $new")  
}
```

49. Delegation with Inline Classes

```
@JvmInline
value class UserId(val value: Int)

class UserRepository {
    private val users = mutableMapOf<Int, User>()

    operator fun getValue(
        thisRef: Any?,
        property: KProperty<*>
    ): User? = users[property.name.hashCode()]
}

val currentUser: User? by UserRepository()
```

50. Combined Delegation Patterns

```
class SmartProperty<T>(
    private var value: T,
    private val validator: (T) -> Boolean,
    private val transformer: (T) -> T
) {
    operator fun getValue(
        thisRef: Any?,
        property: KProperty<*>
    ): T = value

    operator fun setValue(
        thisRef: Any?,
        property: KProperty<*>,
        newValue: T
    ) {
        if (validator(newValue)) {
            value = transformer(newValue)
        }
    }
}
```

```
        }  
    }  
}  
  
var email: String by SmartProperty(  
    value = "",  
    validator = { it.contains("@") },  
    transformer = { it.lowercase() }  
)
```